

COURSE CODE: CSA17

Assessment Tool Weightage: 50%

Scenario Based Assignment

I understand that any violation of academic integrity rules will result in disciplinary action.

of the student: M. SHAKTHI

20
~~20~~ + 18 + 18 + 18 + 18 + 18
20 + 18 + 22 + 18 + 12 = 90
90
SP

Student 17: M SHAKTHI | Register Number: 192312538

1. Scenario

An airline implements an AI system to optimize crew scheduling across hundreds of flights and thousands of crew members. The objective is to minimize crew costs, layover time, and scheduling conflicts while satisfying regulatory rest requirements. The system must continuously adjust assignments as flight delays occur. However, the search space is extremely large, flight schedules change dynamically, and the system may get stuck in locally optimal crew assignments.

Tasks:

Formulate this as an optimization problem and explain why traditional search is inefficient.

- i. Explain how Hill Climbing would work in this scenario and identify its limitations.
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.
- iv. Recommend the best approach and justify your choice based on scalability and adaptability.

2. Scenario

A firefighting robot is deployed inside a burning multi-storey building with an unknown internal layout. It must: navigate safely to locate trapped occupants, avoid hazards (collapsed structures, fire zones), operate with limited visibility due to smoke, and make decisions without a complete floor plan. The robot updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search.
- ii. Analyze the challenges of partial observability and dynamic environments.
- iii. Describe how the robot builds and updates its internal model.
- iv. Compare online search with uninformed and informed search strategies.
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

3. Scenario

An airport is designing an AI system to automatically assign arriving flights to available gates. The system must satisfy multiple constraints: no two flights can occupy the same gate at overlapping times, a limited number of gates support wide-body aircraft, certain airlines have contractual gate preferences, ground crew availability constraints, and special accommodations for delayed connecting flights. The complexity increases as the number of flights and gates grows.

Tasks:

Formulate the problem as a Constraint Satisfaction Problem (CSP).

- i. Clearly define variables, domains, and constraints with explanation.
- ii. Explain how backtracking search works in this scenario.
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.
- iv. Analyze real-world challenges and justify the best strategy for scalability.

Tasks:

i) Hill climbing in crew scheduling:

Hill climbing starts with an initial crew schedule and tries to improve it step by step.

Working:

- 1) Generate an initial valid crew assignment.
- 2) Calculate its cost using the objective function.
- 3) Generate neighbouring solutions by making small changes.
- 4) For example:

- * swap 2 crew members.

- * move a crew member from one flight to another.

Ex: flight A $\rightarrow$ crew 1
flight B $\rightarrow$ crew 2
flight C $\rightarrow$ crew 3

If swapping crew 1, 2 reduces total cost and conflicts, the algorithm accepts the new schedule.

Advantages:

- * simple to implement
- * requires relatively little memory
- * can quickly improve an existing schedule.

Limitations:

- * Local maxima / minima
- * Plateaus
- * Ridges

ii) Local maxima, Plateaus and Ridges:

Local maximum / minimum: A local optimum is a solution that is better than all its immediate neighbours but is not the global optimum.

Real world interpretation: The airline may find a crew schedule that appears very efficient. However, reaching a better schedule may require temporarily accepting a worse assignment before achieving a much better one.

Plateau: A plateau occurs when many neighbouring solutions have the same evaluation value.

Ridge: A ridge occurs when the best direction requires a sequence of moves that cannot be reached through a simple single-step improvement.

(iii) Simulated Annealing and genetic algorithms:

Simulated Annealing: It improves on hill climbing by sometimes accepting a worse solution.

Advantages:

- * can escape local optima
- * can cross plateaus & ridges
- * suitable for dynamic scheduling

Genetic algorithm:

- 1) generate an initial population
- 2) calculate fitness of each schedule
- 3) select better schedules
- 4) perform crossover
- 5) perform mutation

For ex:

flight A $\rightarrow$ C1,
flight B $\rightarrow$ C2,
flight C $\rightarrow$ C3 } Parent 1

flight A $\rightarrow$ C4
flight B $\rightarrow$ C5
flight C $\rightarrow$ C6 } Parent 2

Mutation can randomly change one crew assignment.

(iv) recommended approach: for the airline crew scheduling problem, a genetic algorithm combined with local search or simulated annealing is highly suitable.

A genetic algorithm is especially useful because:

- * The search space is extremely large
- * Many possible crew assignments must be considered.
- * It can explore different regions simultaneously.
- * Mutation helps adapt when flights are delayed / cancelled.
- * Crossover can combine good parts of different schedules.

Why online search is required?

This problem requires online search because the robot does not know the complete environment before starting.

In offline search: the complete map is available

the agent can calculate a plan before acting
the environment is assumed to be sufficient.

Here, the building layout is known

visibility is limited

some paths may be blocked

fire can spread.

structures can collapse

new victims may be detected.

(ii) Challenges of Partial observability & Dynamic environments;

Partial observability: The robot cannot see the complete environment.

Ex: Smoke blocks cameras

walls hide rooms

sensors have limited range

victims may be behind obstacles.

Dynamic environment: The environment changes continuously.

Ex: fire spreads

walls or ceilings may collapse

temperature changes

smoke density changes, emergency exits may become blocked.

Main challenges:

1) incomplete information

2) sensor uncertainty

3) changing hazards

4) limited battery

5) limited communication

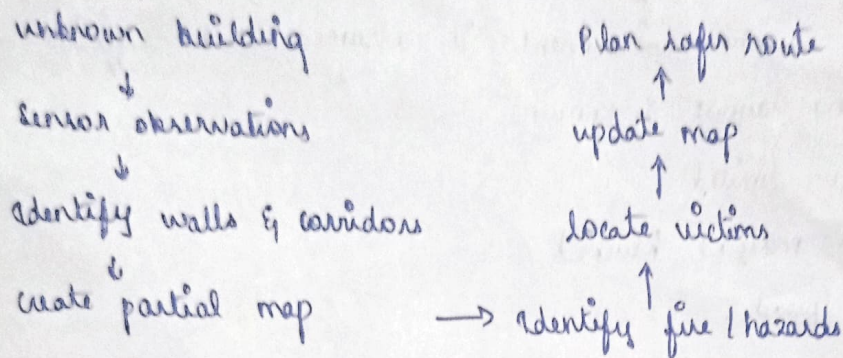
6) real-time decision making

7) safety of the robot & victims

(ii) Building & updating the 'internal model';

the robot can create an internal map/model using sensors such as: camera, LiDAR, thermal sensors, gas sensors, temperature sensors, distance sensors.

for example:



(iv) Comparison with uninformed and informed search:

uninformed search: uninformed algorithms do not have much knowledge about which direction is better.

Ex: Breadth first search (BFS)

depth first search (DFS)

uniform cost search (UCS)

Informed search: informed algorithms use a heuristic to guide search.

(v) Most suitable approach:

The robot should:

1) Sense the environment

2) Build a partial map

3) Identify hazards

4) Estimate safe paths

5) Move toward promising areas

6) Search for victims

7) update the map

8) Re-plan whenever new information is received.

n variables, Domain & Constraints:

variables: let each arriving flight be a variable.

for ex:

$f-1 = \text{flight 1}$

$f-2 = \text{flight 2}$

$f-3 = \text{flight 3}$

Each variable represents the gate assigned to that flight.

Domains: the domain of each flight contains the gates that can potentially be assigned.

for ex: $[D(f-1) = \{G_1, G_2, G_3, G_4\}]$

ex: $[D(f-1) = \{G_1, G_3\}]$

if only G_1 & G_3 can accommodate that aircraft.

(ii) Backtracking Search: it is a systematic method for solving CSPs.

Steps: 1) Select an unassigned flight

2) select a possible gate

3) check whether the assignment satisfies constraints.

4) if valid, continue with the next flight.

5) if invalid, try another gate.

6) if no gate works, go back to the previous flight.

7) change its assignment & continue

Basic backtracking algorithm:

BACKTRACKING (assignment):

if all flights are assigned:

return assignment

select an unassigned flight

for each gate in its domain:

if gate satisfies all constraints:
assign gate to flight

result = BACKTRACKING (assignment)

if result is successful:

return result

remove assignment

return failure

(iii) Constraint propagation: It reduces the search space before and during backtracking. It removes values from a variable's domain when those values can't lead to a valid solution.

Ex: Suppose: $[D(F1) = \{g1, g2\}]$, $[D(F2) = \{g1, g2, g3\}]$

If $F1$ is assigned $g1$ and the flights overlap, then $g1$ is removed from $F2$'s domain: $[D(F2) = \{g2, g3\}]$

(iv) Real world challenges and best strategy for scalability:

Airport gate assignment becomes difficult when the number of flights and gates increases.

Major challenges:

- 1) large number of flights
- 2) limited number of gates
- 3) overlapping flight schedules
- 4) Aircraft size differences
- 5) Airline preferences
- 6) Ground crew availability
- 7) Flight delays and cancellations
- 8) Connecting passenger requirements
- 9) International / domestic terminal restrictions
- 10) Emergency gate requirements

Best strategy:

- 1) Backtracking
- 2) Forward checking
- 3) MRV
- 4) Degree Heuristic
- 5) Least Constraining value
- 6) Soft Constraints